

Scheduling Weight Transitions for Quantization-Aware Training

Junghyup Lee^{1,*}

Jeimin Jeon^{2,3,*}

Dohyung Kim⁴

Bumsub Ham^{2,†}

¹ Samsung Research

²Yonsei University

³Articron Inc.

⁴ Samsung AI center

<https://cvlab.yonsei.ac.kr/projects/TRS/>

Abstract

Quantization-aware training (QAT) simulates a quantization process during training to lower bit-precision of weights/activations. It learns quantized weights indirectly by updating latent weights, i.e., full-precision inputs to a quantizer, using gradient-based optimizers. We claim that coupling a user-defined learning rate (LR) with these optimizers is sub-optimal for QAT. Quantized weights transit discrete levels of a quantizer, only if corresponding latent weights pass transition points, where the quantizer changes discrete states. This suggests that the changes of quantized weights are affected by both the LR for latent weights and their distributions. It is thus difficult to control the degree of changes for quantized weights by scheduling the LR manually. We conjecture that the degree of parameter changes in QAT is related to the number of quantized weights transiting discrete levels. Based on this, we introduce a transition rate (TR) scheduling technique that controls the number of transitions of quantized weights explicitly. Instead of scheduling a LR for latent weights, we schedule a target TR of quantized weights, and update the latent weights with a novel transition-adaptive LR (TALR), enabling considering the degree of changes for the quantized weights during QAT. Experimental results demonstrate the effectiveness of our approach on standard benchmarks.

1. Introduction

Recent neural networks use wide and deep architectures [12, 17, 19, 42, 45] requiring millions of parameters and computations. Network quantization converts full-precision weights and/or activations into low-bit ones. This reduces storage usage and computational overheads drastically, but the low-bit models perform worse than the full-precision ones. To alleviate this problem, many approaches [5, 10, 21, 32, 37] adopt quantization-aware training (QAT) that simulates a quantization process during training. It is not straightforward to optimize discrete

quantized weights using gradient-based optimizers (e.g., stochastic gradient descent (SGD)) with continuous gradients. QAT instead exploits full-precision latent weights and a quantizer involving a discretization function (e.g., a round function) to update the quantized weights indirectly.

QAT mainly consists of three steps: (1) In a forward propagation step, full-precision latent weights are converted to quantized weights using a quantizer, and the quantized weights are used to compute an output; (2) Gradients w.r.t an objective function are then back-propagated to the latent weights in a backward propagation step; (3) In an optimization step, the latent weights are updated with the gradients. Previous works mostly focus on the forward and backward propagation steps by designing quantizers [4, 5, 10, 21, 27, 37, 46, 48, 49] and addressing a vanishing gradient problem, caused by the discretization function in a quantizer [1, 11, 22, 26, 47], respectively. They do not pay attention to the optimization step, and simply exploit gradient-based optimizers with a user-defined learning rate (LR), such as SGD or Adam [23], to update the latent weights. This optimization strategy is, however, designed for training full-precision models, and does not consider how quantized weights are changed, which is sub-optimal for QAT.

When training a full-precision model with a gradient-based optimizer, we typically decay a LR progressively to update full-precision weights in a coarse-to-fine manner, which guarantees the convergence of the model [18, 24]. That is, we can control the degree of weight changes manually by scheduling the LR, since the magnitude of a single parameter change, so-called an *effective step size* [23], is highly correlated with the LR (Fig. 1a vs. Fig. 1b). For example, an average effective step size, which quantifies the degree of changes in weights, for a small LR is lower than that for a large one. We have found that this does not hold, when the optimizers coupled with a LR are applied to a quantized model in QAT. Since QAT updates quantized weights indirectly from latent weights and a quantizer, an average effective step size for quantized weights is less correlated with a LR for latent weights (Fig. 1a vs. the blue curve in Fig. 1c). To be more specific, different from full-

*Equal contribution. †Corresponding author.

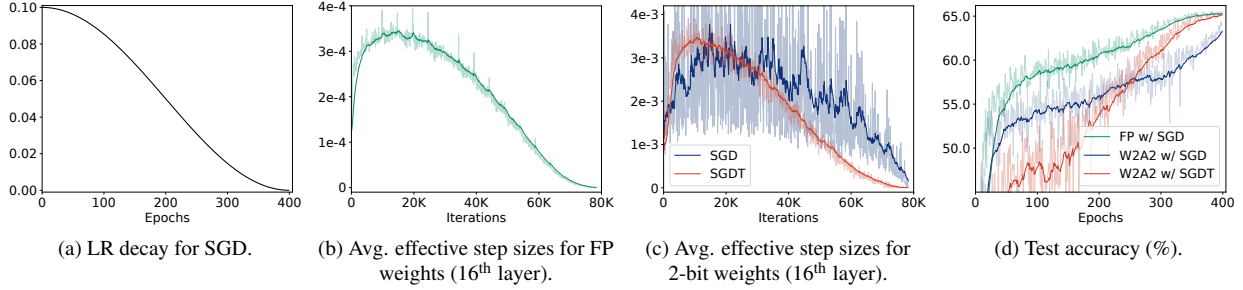


Figure 1. Training curves of full-precision (FP) and quantized models for ResNet-20 [12] on CIFAR-100 [25]. Both weights (W) and activations (A) are quantized to a 2-bit precision (W2A2). With a gradient-based optimizer (SGD), we can control the average effective step size of FP weights roughly by scheduling a LR ((a) vs. (b)), while we could not for quantized weights (the blue curve in (c)). The curve for quantized weights is noisy, and decreases rapidly at the end of training, suggesting that 1) the quantized weights can alter significantly with a small LR and/or a small change of a LR, disturbing a coarse-to-fine parameter update and causing an unstable training, and 2) adopting a manually scheduled LR for QAT is sub-optimal. The optimizer coupled with our scheduling technique (SGDT) can control the average effective step size of quantized weights by adjusting the number of transitions explicitly (the red curve in (c)), showing better results in terms of accuracy and convergence (the red curve in (d)).

precision weights, quantized weights change their discrete levels, only when corresponding latent weights pass transition points, where the quantizer alters discrete states (we call this as *transitions*). This makes it hard to control an average effective step size of quantized weights by adjusting a LR, disturbing coarse-to-fine parameter updates and the convergence of a model, even at the end of training (the blue curve in Fig. 1d). For example, if latent weights are concentrated around a transition point, they can pass the point easily with a tiny LR, inducing significant changes of quantized weights.

In this paper, we introduce a transition rate (TR) scheduling technique for QAT, which allows to update latent weights w.r.t the transitions of quantized weights explicitly. We define the TR of quantized weights as the number of quantized weights changing discrete levels at each iteration for optimization, divided by the total number of the weights. Note that an effective step size of each quantized weight is either zero or a discrete value (*i.e.*, a distance between two quantization levels), indicating that an average effective step size for quantized weights is mainly affected by the number of transitions. We thus conjecture that the number of transitions, or similarly, the TR is a key for controlling the degree of parameter changes for quantized weights. Based upon this, we propose to schedule a target TR of quantized weights, instead of a LR for latent weights, and update the latent weights with a novel transition-adaptive learning rate (TALR) to adjust a TR of quantized weights accordingly. The TALR is changed adaptively to match the current TR of quantized weights with the target one. By scheduling the target TR, we are able to adjust the average effective step size of quantized weights (the red curve in Fig. 1c). This allows us to optimize quantized weights in a coarse-to-fine manner, and provides a stable training process (the red curve in Fig. 1d). To the best of our knowledge, scheduling a TR for QAT has not

been explored, instead of a LR as in full-precision training. We demonstrate the superiority of our TR scheduling technique over the plain LR scheduling using various network architectures [12, 30, 32, 38, 44] and optimizers (*e.g.*, SGD, Adam [23], and AdamW [34]) on image classification [7, 25] and object detection [29]. In summary, the main contributions of our work are threefold:

- We claim the necessity of a training scheduler specialized for general QAT for the first time, where quantized weights are optimized indirectly by latent weights and a quantizer. To update latent weights in QAT, we propose to focus on the changes of quantized weights used for computing an output of a quantized network, which has not been covered by plain optimizers using a user-defined LR.
- We present a novel TR scheduling technique for QAT together with a TALR, which is adjusted considering a TR of quantized weights, controlling an average effective step size of quantized weights accordingly.
- We demonstrate the effectiveness and generalization ability of our approach on network quantization, boosting the performance of various models consistently.

2. Related Work

QAT. QAT methods simulate quantization during training by converting full-precision latent weights into quantized ones. Early works adopt fixed quantizers for binary [37, 49], ternary [27], and multi-bit [4, 49] representations, attempting to minimize quantization errors. Recent approaches introduce trainable quantizers that learn quantization parameters, such as intervals [5, 10, 21] or non-uniform levels [46, 48], significantly improving performance. These methods rely on the straight-through estimator (STE) [2] to handle non-differentiable quantization, but suffer from gradient mismatch [22, 47]. To address this, several works avoid exploiting the STE by using differentiable quantizers [11, 22, 47] or adjusting the gradients depending on the

latent weights [1, 26]. Despite advancements, they overlook the fact that quantized weights act differently from full-precision counterparts, and simply adopt the same optimization strategies for full-precision models, that exploit manual LR scheduling techniques (*e.g.*, step decay or cosine annealing [33]), to train a quantized model. We highlight that quantized weights are updated indirectly by full-precision latent weights and a quantizer. Based on this, we propose to focus on actual changes of quantized weights, and present a TR scheduling technique specialized for QAT.

Recently, the work of [35] points out that quantized weights tend to oscillate between adjacent quantization levels during QAT. To address the oscillation problem, it proposes to freeze latent weights during training or to incorporate a regularization term into the objective function. While this approach could alleviate the oscillations, it still relies on the conventional optimization method using a LR, and thus it cannot control the average effective step size of quantized weights explicitly. Moreover, hyperparameters are chosen carefully according to network architectures and bit-widths, since freezing weights or adding the regularization term could disturb the training process. We have observed that it is difficult to control the average effective step size of quantized weights with the LR scheduling, and the reason for this is closely related to the oscillation problem. Different from [35] that reduces oscillations themselves by freezing or regularizing the weights, we attempt to control the actual change of quantized weights by scheduling a target TR. In this way, we can achieve consistent performance gains under the various bit-width settings with the same set of hyperparameters for each architecture, which is not feasible for [35].

Most recently, pseudo-quantization training (PQT) [6, 39, 40] addresses oscillations by injecting pseudo-quantization noise into full-precision weights, instead of exploiting the quantization operations during training. While PQT stabilizes training, it introduces discrepancies between training and inference, as the actual quantization operations are not considered during training, leading to suboptimal performance. To overcome this, they leverage auxiliary techniques like knowledge distillation or mixed-precision quantization, but at the cost of computational overheads. In contrast, our method mitigates oscillations directly through TR scheduling, maintaining consistency between training and inference with minimal overhead, leading to consistent performance gains over plain optimization methods, with negligible overheads.

Optimization Methods. Neural networks are generally trained using a gradient-based optimizer coupled with a LR scheduling technique. Gradient-based optimizers update network weights based on the gradients w.r.t an objective function. SGD is a vanilla optimizer for minimizing the objective function, which often exploits the first moment of

the gradients to accelerate update steps near local optima. Many works [9, 23, 43] propose advanced optimizers using adaptive gradients. They accumulate squared gradients using a historical sum [9] or a running average [23, 43] (*i.e.*, the second moment of gradients), which are then used for normalizing each gradient dimension adaptively. This enables highlighting infrequent features at training time, which is particularly useful for sparse gradients [9]. All of these optimizers exploit a LR to update full-precision weights. They typically use a large LR initially and decay it to a small value by a LR scheduling technique, such as step decay or cosine annealing [33]. The large initial LR encourages weights to explore local optima in a loss space, and the small LR at the end of training prevents the weights from overshooting from a local optimum [18, 24]. A recent work [28] also points out that the LR scheduling is important for both generalization and performance, since large and small LR play complementary roles in memorizing different types of patterns. The aforementioned optimization strategies are, however, designed for training a full-precision model, directly updating full-precision weights used for inference. For QAT, quantized weights are used for forward/backward passes, but full-precision latent weights are instead updated to train the quantized weights. We propose to consider this unique property of QAT to optimize the latent weights, and introduce a novel TR scheduling technique, specially designed for network quantization.

Closely related to ours, few works [13, 41] present optimizers for binary networks that train binary weights directly, rather than using latent weights. To this end, they adopt the first moment of gradients [13] or its variant using the second moment [41] to flip binary values via thresholding. Although these methods do not use latent weights, the first moment and its variant have a role similar to the latent weights in that both accumulate gradients, and they schedule a threshold manually for flipping binary values. This suggests that they do not consider the degree of parameter changes in the binary weights explicitly, causing the same problem as the conventional optimizers using a LR. These methods are also applicable for binary networks only. On the contrary, our method adjusts a TR explicitly to control an average effective step size of quantized weights consequently, and it can be applied to general QAT, including both binary and multi-bit quantization schemes.

3. Preliminary

QAT inserts weight and/or activation quantizers into each layer of a neural network to simulate a quantization process at training time. Here we briefly describe a quantizer and an optimizer in QAT.

Quantizer. Weight and activation quantizers take full-precision latent weights and activations in a layer, respectively, and produce low-bit representations. Here we mainly

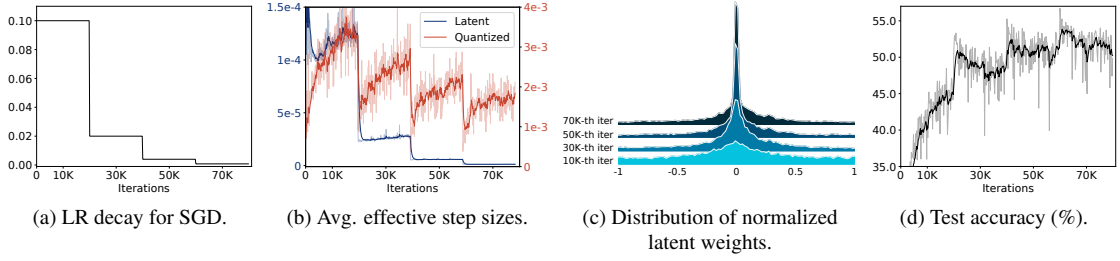


Figure 2. Empirical analysis on QAT using SGD with a step LR decay. We binarize both weights and activations of ResNet-20 [12] and train the model on CIFAR-100 [25]. For the visualizations in (b) and (c), we track the latent and quantized weights in the 16th layer. We can see that the average effective step size of latent weights (the blue curve in (b)) is controlled by the LR in (a), while that for the quantized weights changes significantly even with a small LR (the red curve in (b)). This is because the change of quantized weights is also affected by the distribution of latent weights approaching the transition point (*i.e.*, zero in (c)). The large changes in the quantized weights at the end of training (the red curve in (b)) degrade the performance in (d). (Best viewed in color.)

explain the weight quantizer. The activation quantizer is similarly defined. Let us denote by $\mathbf{w}$ full-precision latent weights. The quantizer first normalizes and clips the latent weights to adjust their range:

$$\mathbf{w}_n = f(\mathbf{w}), \quad (1)$$

where we denote by $\mathbf{w}_n$ normalized weights. f is a normalization function involving scaling and clipping operations, which can be either hand-designed [49] or be trainable [5, 10, 21]. The normalized weights $\mathbf{w}_n$ are then converted to discrete ones $\mathbf{w}_d$ using a discretization function g :

$$\mathbf{w}_d = g(\mathbf{w}_n). \quad (2)$$

The discretization function g is typically a signum or a round function for binary or multi-bit quantization schemes, respectively. Note that STE [2] is usually adopted in a backward pass to avoid a vanishing gradient problem, caused by the discretization function, propagating the same gradient from $\mathbf{w}_d$ to $\mathbf{w}_n$. Lastly, the quantizer produces quantized weights $\mathbf{w}_q$ by de-normalizing the discrete weights $\mathbf{w}_d$:

$$\mathbf{w}_q = h(\mathbf{w}_d), \quad (3)$$

where h is a de-normalization function for post-scaling. The de-normalization could possibly be omitted (or fixed) when the quantized layer is followed by a normalization layer (*e.g.*, batch normalization [20]), since it imposes the scale invariance to the weights and activations [14, 15], suggesting that de-normalization has no effect on either the forward or backward pass.

Optimizer. In QAT, the latent weights $\mathbf{w}$ are updated, instead of optimizing the quantized weights $\mathbf{w}_q$ directly. That is, updating the latent weights in turn alters the quantized ones during training. More specifically, the quantized weights change their discrete levels if corresponding normalized latent weights $\mathbf{w}_n$ pass transition points of the discretization function g (*e.g.*, zero for the signum function) after updating the latent weights $\mathbf{w}$. Previous works typically use gradient-based optimizers with a user-defined LR to update the latent weights as follows:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \mu^t \mathbf{g}^t, \quad (4)$$

where the superscript t indicates an iteration step, and we denote by $\mathbf{g}$ and μ a gradient term and the LR, respectively. Note that the gradient term $\mathbf{g}$ is computed differently depending on the types of optimizers. For example, SGD uses the first moment of gradients.

4. Method

In this section, we first present a detailed analysis of a conventional optimization method using a manually scheduled LR in the context of QAT (Sec. 4.1). We then introduce a novel TR scheduling technique (Sec. 4.2).

4.1. Empirical analysis

Conventional optimizers use a LR decay technique when training a full-precision model. They update model parameters gradually in a coarse-to-fine manner, which encourages a model to find a better local optimum in a loss space, and prevents overshooting from a local optimum [18, 24]. This suggests that the optimizers control an average effective step size (*i.e.*, the degree of parameter changes) of full-precision weights by adjusting the LR. We have empirically found that this does not hold for QAT. Namely, the average effective step size of quantized weights in QAT is hardly controlled by a conventional LR scheduling technique in gradient-based optimizers.

To understand this problem in detail, we show an empirical analysis on 1) how a gradient-based optimizer, coupled with a manually scheduled LR, changes latent and quantized weights within a framework of QAT, and 2) the influence of the changes on the classification accuracy of a quantized model (Fig. 2). We train ResNet-20 [12] with binary weights and activations on CIFAR-100 [25] using a SGD optimizer with a step LR decay method. We can see from Fig. 2a and the blue curve in Fig. 2b that the average effective step size of latent weights is controlled by a LR, which is consistent with the result in a full-precision model (*e.g.*, Fig. 1a vs. Fig. 1b). The reason is that the latent weights in

QAT and the weights in a full-precision model are continuous values, and the LR is responsible directly for updating the weights, *e.g.*, as in Eq. (4). On the contrary, quantized weights alter significantly, even with a small LR (the red curve in Fig. 2b). Since QAT uses quantized weights in a forward propagation step to compute gradients w.r.t an objective function, the large changes of quantized weights at the end of training make a training process unstable, disturbing a quantized model to converge (Fig. 2d).

To delve deeper into this problem, let us suppose that a quantized weight needs to alter its discrete level (*e.g.*, from a negative value to a positive one in the binary quantization) in order to minimize a training loss. A corresponding latent weight then keeps accumulating gradients to move towards a transition point, and once a transition occurs, the latent weight might stay near the transition point. We can observe in Fig. 2c that the normalized latent weights (*i.e.*, w_n in Eq. (1)) are approaching the transition point (*i.e.*, zero in this case) progressively according to the number of iterations. The quantized weight is hence likely to oscillate between adjacent discrete levels with small LR in later training iterations (see the high peak at the 70K-th iteration in Fig. 2c). This coincides with the recent finding in [35, 36] that the quantized weights tend to oscillate during QAT, making it difficult to stabilize the batch normalization statistics [20], and degrading the performance at test time. This analysis indicates that 1) the average effective step size of quantized weights is largely affected by the distribution of latent weights, and 2) the reason why the LR is not a major factor for controlling the average effective step size in QAT, contrary to an optimization process of a full-precision model, is that the quantized weight alters only when the latent weight passes a transition point of a quantizer, but the LR cannot adjust the number of transitions explicitly. Consequently, our empirical analysis suggests the necessity of a training scheduler specific to QAT that allows to update latent weights adaptively considering the transitions in quantized weights.

4.2. TR scheduler

Here we present a relationship between an effective step size and transitions in quantized weights, and describe our approach to TR scheduling in a single layer.

TR of quantized weights. We say that a transition occurs if a latent weight passes a transition point of a quantizer after a single update. The number of transitions is hence equal to that of quantized weights changing discrete levels after the update. We can count the number of transitions by observing whether discrete weights (*i.e.*, w_d in Eq. (2)) are changed or not after the update. Here we focus on a TR, the number of transitions divided by the total number

of quantized weights, defined as follows:

$$k^t = \frac{\sum_{i=1}^N \mathbb{I}[w_d^t(i) \neq w_d^{t-1}(i)]}{N}, \quad (5)$$

where we denote by k^t and $w_d^t(i)$ the TR and the i -th element of discrete weights at the t -th iteration step, respectively, and N is the total number of quantized weights. $\mathbb{I}[\cdot]$ is an indicator function that outputs one if a given statement is true and zero otherwise.

Relation between an effective step size and a transition.

An *effective step size* [23] indicates the magnitude of a single parameter change. We can compute the effective step size of a quantized weight w_q by measuring its absolute difference before and after a single update as follows:

$$|\Delta w_q^t| = |w_q^t - w_q^{t-1}|, \quad (6)$$

where we denote by $|\Delta w_q^t|$ an effective step size of the quantized weight at the t -th iteration step. We will show that the effective step size is related to a transition of the quantized weight. Let us denote by δ^t a post-scaling factor of the de-normalization function h in Eq. (3) at the t -th iteration step. If the discretization function g in Eq. (2) is a rounding function for multi-bit quantization (*i.e.* a discrete weight w_d^t is an integer value), we can rewrite Eq. (6) as follows:

$$|\Delta w_q^t| = |\delta^t w_d^t - \delta^{t-1} w_d^{t-1}|. \quad (7)$$

If g is a signum function (*i.e.*, $w_d^t \in \{-1, 1\}$) for binary quantization, Eq. (6) can be represented as follows:

$$|\Delta w_q^t| = \frac{1}{2} |\delta^t w_d^t - \delta^{t-1} w_d^{t-1}|. \quad (8)$$

Note that the change of δ^t in a single update is typically small (*i.e.*, $\delta^t \approx \delta^{t-1}$) or we can set the post-scaling factor δ^t as a constant value if the quantized layer is followed by a normalization layer [14, 15] (*e.g.*, as in [26]). Assuming that the change of δ^t is negligible within a single update and a latent weight passes a single transition point when a transition occurs, we can approximate the effective step size of the quantized weight as follows:

$$|\Delta w_q^t| \approx \delta^t \mathbb{I}[w_d^t \neq w_d^{t-1}]. \quad (9)$$

That is, the effective step size of the quantized weight is at most δ^t if a transition occurs, and zero otherwise. This indicates that individual effective step sizes of quantized weights are discrete values (*i.e.*, zero or δ^t) determined by the quantizer. Note that the effective step size for each full-precision weight can be adjusted by a LR, since the weight is a continuous value, which is however not applicable for the quantized weight changing discretely. Accordingly, adjusting the number of transitions, or equivalently a TR, is

important to control an average effective step size of quantized weights. Based upon this, we design a TR scheduling technique adjusting a TR of quantized weights explicitly, allowing us to control the degree of parameter changes in the quantized weights accordingly.

TR scheduler. We incorporate our TR scheduling technique into an optimization process by introducing a transition-adaptive learning rate (TALR) to update latent weights, allowing to adjust a TR of quantized weights manually, w.r.t a target TR. To this end, we mainly apply three operations at every iteration: Estimating a running TR using a momentum estimator, adjusting a TALR w.r.t a target value, and updating latent weights. Specifically, we first compute a running TR of quantized weights for each iteration t using an exponential moving average with a momentum of m :

$$K^t = mK^{t-1} + (1 - m)k^t, \quad (10)$$

where we denote by K^t a running TR. Motivated by the running statistics in *e.g.*, batch normalization [20], we use the momentum estimator to obtain the running TR, which roughly averages the TRs over recent training iterations, instead of using the TR, k^t in Eq. (5), directly. This allows us to use a stable statistic of the TR, and alleviates the influence from outliers. We then adjust a TALR based on the running TR K^t and a target one:

$$U^t = \max(0, U^{t-1} + \eta(R^t - K^t)), \quad (11)$$

where we denote by U^t and R^t the TALR and the target TR at the iteration step t , and η is a hyperparameter controlling the extent of the TALR update. Note that we can schedule the target TR R^t using typical schedulers (*e.g.*, step decay), which is analogous to the LR scheduling technique. With the TALR U^t at hand, we update the latent weights $\mathbf{w}^t$ as follows:

$$\mathbf{w}^{t+1} = \mathbf{w}^t - U^t \mathbf{g}^t, \quad (12)$$

where $\mathbf{g}^t$ is a gradient term computed depending on the type of an optimizer (*e.g.*, the first moment of gradients in SGD). Updating the latent weights $\mathbf{w}^t$ with the TALR U^t enables controlling the running TR of quantized weights K^t w.r.t the target TR R^t . For example, if a current running TR K^t is smaller than the target one R^t , the TALR U^t increases according to Eq. (11). The latent weights in Eq. (12) are then updated largely, compared to the previous iteration. This encourages more latent weights to pass transition points of a quantizer, which in turn raises the TR in the next step. Similarly, in the opposite case, the TALR decreases to reduce the TR. Note that one can adjust the TALR in a different way from Eq. (11) while achieving the same effect, and we discuss the variants of update algorithms for TALR in the Sec. S3.2 of the supplement. Our approach connects the latent and quantized weights, in contrast to conventional

optimization methods, making it possible to control an average effective step size of quantized weights via scheduling a target TR.

4.3. Quantization scheme

We apply the TR scheduler to QAT with various bit-width settings, including binary and multi-bit representations. In the following, we describe quantization schemes used in our experiments.

Multi-bit quantization. We modify LSQ [10], the state-of-the-art method for multi-bit uniform quantization¹. We define our b -bit quantizer as follows:

$$\mathbf{x}_q = \frac{1}{\gamma} \left\lceil \text{clip} \left(\frac{\gamma \mathbf{x}}{s}, \alpha, \beta \right) \right\rceil, \quad (13)$$

where $\mathbf{x}_q$ is an output of the quantizer. We denote by $\mathbf{x}$ an input to the quantizer, which can be either latent weights or input activations. $\text{clip}(\cdot, \alpha, \beta)$ is a clipping function with lower and upper bounds of α and β , respectively, and $\lceil \cdot \rceil$ is a round function. Following LSQ, we employ a learnable scale parameter s for each quantizer, adjusting the range of quantization interval². We set the bit-specific constants (α, β, γ) as $(-2^{b-1}, 2^{b-1} - 1, 2^{b-1})$ and $(0, 2^b - 1, 2^b)$ for weight and activation quantizers, respectively. We do not perform a post-scaling with the learnable scale parameter s after the round function in contrast to LSQ. That is, we fix the output range of a quantizer, enforcing the output of the quantizer $\mathbf{x}_q$ to be fixed-point numbers, regardless of the range of an input $\mathbf{x}$, which is more suitable for hardware implementation. Note that the scale difference between the input and output of a quantizer does not matter if each convolutional/fully-connected layer is followed by a normalization layer (*e.g.*, batch normalization [20]), imposing the scale invariance after every quantized layer [14, 15]. This ensures that post-scaling does not affect either the forward or backward pass. When the normalization is not used, we optionally apply a learnable post-scaling technique to outputs of convolutional/fully-connected layers [26].

Binary quantization. We apply two binarization methods. First, we use the network architecture of ReActNet [32] and its quantization scheme, which is the state of the art on binary quantization. ReActNet modifies the ResNet [12] or MobileNet-V1 [16] architectures by adopting the Bi-Real structure [31] that adds more residual connections, while exploiting real-valued 1×1 convolutions in

¹Using the same network architecture (*i.e.*, a vanilla version of ResNet), our modifications provide similar or better baseline results on ImageNet [7], compared to the performance of LSQ, reproduced in [3].

²We train scale parameters in activation quantizers only, and do not train them in weight quantizers, when the TR scheduling technique is adopted. Otherwise, transitions could occur, even when the latent weights are not updated. For a fair comparison, we use learnable scale parameters for weight quantizers, when using plain optimizers without TR scheduling. See the Sec. S5.2 of the supplement for details.

Table 1. Quantitative comparison of quantized models on ImageNet [7] in terms of a top-1 validation accuracy. We train quantized models with plain optimization methods (SGD and Adam [23]) or ours using a TR scheduler (SGDT and AdamT). The bit-widths of weights (W) and activations (A) are represented in the form of W/A. For comparison, we report the performance of full-precision (FP) and activation-only binarized (W32A1) models. The results of ReActNet-18 [32] for the plain optimizers are reproduced with an official source code.

Optimizer	MobileNetV2 (FP: 71.9)			ReActNet-18 (W32A1: 66.8)		ResNet-18 (FP: 69.9)			
	2/2	3/3	4/4	1/1	1/1	2/2	3/3	4/4	4/4
SGD	46.9	65.6	69.9	65.0	55.3	66.8	69.5	70.5	
SGDT	53.6	67.0	70.5	65.3	55.8	66.9	69.7	70.6	
Adam	49.6	66.5	70.0	65.3	56.1	66.7	69.5	70.1	
AdamT	53.8	67.3	70.8	65.7	56.3	67.2	69.7	70.4	

Table 2. Quantitative comparison of quantized models on CIFAR-100/10 [25] in terms of a top-1 test accuracy.

Optimizer	CIFAR-100				CIFAR-10			
	ReActNet-18 (W32A1: 69.6)		ResNet-20 (FP: 65.1)		ReActNet-18 (W32A1: 91.3)		ResNet-20 (FP: 91.1)	
	1/1	1/1	2/2	2/2	1/1	1/1	2/2	2/2
SGD	69.7	54.9	64.1	64.1	90.9	85.2	90.2	90.2
SGDT	72.2	55.8	65.5	65.5	93.0	85.6	90.7	90.7
Adam	69.5	54.8	63.3	63.3	90.4	84.8	90.2	90.2
AdamT	71.8	55.9	65.2	65.2	92.9	85.7	91.1	91.1

Table 3. Quantitative comparison of quantized models on ImageNet [7] in terms of a top-1 validation accuracy. We train quantized models with plain optimization method (AdamW [34]) or ours using a TR scheduler (AdamWT).

Optimizer	DeiT-T (FP: 72.0)		DeiT-S (FP: 79.9)	
	2/2	3/3	2/2	3/3
AdamW	54.6	68.1	68.4	77.6
AdamWT	57.4	69.5	71.8	78.5

the residual connections. This approach also uses learnable shift operations before quantization and activation functions. Second, we binarize vanilla ResNet models to compare binary and multi-bit quantization schemes under a fair training setting. To this end, we design a binary quantizer using Eq. (13). For a weight quantizer, we set α , β , and γ , to -1, 1, and 1, respectively, and replace the round operator with a signum function to obtain a binary value of -1 or 1. For an activation quantizer, we set those values as 0, 1, and 1, respectively, to generate a binary activation of 0 or 1.

5. Experiments

We describe our experimental settings (Sec. 5.1) and show results on image classification and object detection (Sec. 5.2). We then analyze the TR scheduling technique (Sec. 5.3). More detailed analyses and discussions are provided in the supplement.

Table 4. Quantitative results on object detection. We train RetinaNet [30] on the training split of MS COCO [29] using either the plain optimization method (SGD) or ours (SGDT). We report the average precision (AP) on the validation split.

Backbone	W/A	Optimizer	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
ResNet-50	FP	SGD	37.80	57.62	40.50	23.12	41.39	49.70
		SGDT	38.05	57.75	40.23	22.51	41.46	49.68
	4/4	SGD	38.36	58.01	40.76	22.46	41.87	49.71
		SGDT	38.36	58.01	40.76	22.46	41.87	49.71
	3/3	SGD	37.32	56.87	39.71	21.90	40.82	48.97
		SGDT	37.59	56.89	40.18	21.51	40.98	49.07

5.1. Experimental settings

For image classification, we train quantized models for MobileNetV2 [38], ResNet families [12], ReActNet-18 [32], and DeiT-T/S [44] on CIFAR-10/100 [25] and/or ImageNet [7]. We train them using a cross-entropy loss, except for ReActNet-18 on ImageNet, where we use a distributional loss [32] following the work of [32]. For object detection, we adopt RetinaNet [30] with ResNet backbones on MS COCO [29]. Unlike the previous QAT methods [46, 50], we use a shared prediction head to handle features of different resolutions, analogous to the original RetinaNet [30]. For ease of activation quantization, we add a ReLU layer after each convolutional layer in the prediction head, so that all inputs of activation quantizers are non-negative. For more details, please refer to the Sec. S4.1 of the supplement.

While our method requires additional computations (*i.e.*, element-wise comparison in Eq. (5) and scalar operations in Eqs. (10)-(11)), they are computationally cheap compared to the whole training process. The training time increases by only 2% compared to the plain optimization methods with the same machine (Sec. S2.5 of the supplement).

5.2. Results

Image classification. We provide in Tables 1-3 quantitative comparisons of quantized models trained with optimizers using plain optimization methods and our approach. We report a top-1 classification accuracy on ImageNet [7] and CIFAR-100/10 [25] using the MobileNetV2 [38], ReActNet-18 [32], ResNet-18/20 [12], and DeiT-T/S [44] architectures. From these tables, we observe three things: (1) Our method provides substantial accuracy gains over the plain optimizers, regardless of the datasets, network architectures, and quantization bit-widths. This indicates that scheduling a target TR is a better choice for the optimization process in QAT compared to the conventional strategy scheduling a LR. (2) The performance gaps on ImageNet using light-weight MobileNetV2 (0.6~6.7%) are more significant than the ones using ReActNet-18 or ResNet-18 (0.1~0.5%). Moreover, the performance gaps become larger for smaller bit-widths of MobileNetV2. These results suggest that the TR scheduling technique is especially useful for compressing networks aggressively, such as quantiz-

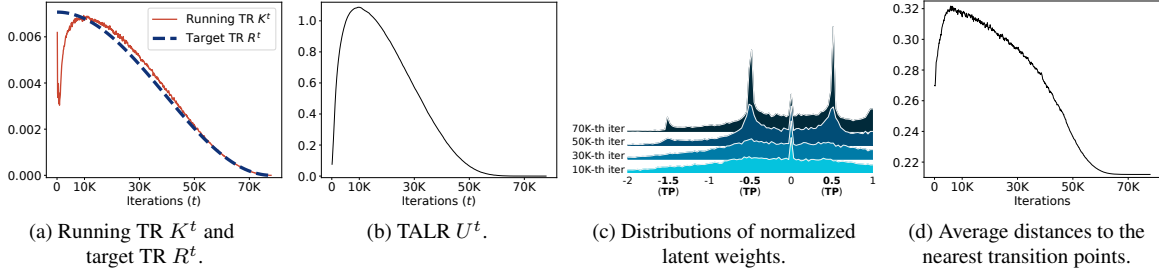


Figure 3. Analysis on TR scheduling. We train ResNet-20 [12] on CIFAR-100 [25] using SGDT, where we quantize both weights and activations with 2-bit representations. We visualize distributions of normalized latent weights in the 16th layer in (c), and average distances between normalized latent weights and the nearest transition points in (d). The transition points in (c) are denoted by TPs in the x-axis. The top-1 test accuracy and average effective step sizes of quantized weights are shown by the red curves in Figs. 1d and 1c, respectively.

ing a light-weight model or extremely low-bit quantization. (3) Considering the results for ReActNet-18 and the ResNet families, our approach outperforms the conventional optimization methods by significant margins (0.4~2.5%) on the small dataset (*i.e.*, CIFAR-100/10). On the large-scale dataset (*i.e.*, ImageNet), it also shows superior results, achieving 0.1~0.5% accuracy gains. The overall performance gaps decrease on ImageNet, possibly because the plain optimizers with a gradually decaying LR (*e.g.*, cosine annealing LR [33]) benefit from lots of training iterations on ImageNet (roughly 600K). They, however, do not show satisfactory results within a small number of iterations on CIFAR-100/10 (roughly 80K), compared to ours.

Object detection. We compare in Table 4 the quantization performance of detection models in terms of an average precision (AP) on the validation split of MS COCO [29]. We train RetinaNet [30] with the ResNet-50 [12] backbone using either SGD or SGDT on the training split of MS COCO. We can observe in Table 4 that the TR scheduling technique boosts the AP consistently over the SGD baselines across different bit-widths, similar to the results on image classification. This suggests that the TR scheduling technique is also useful for the object detection task involving both regression and classification, demonstrating once more the effectiveness of our method and its generalization ability to various tasks. Additional quantitative results on object detection with different backbone networks (*e.g.*, ResNet-18/34) and qualitative results are provided in the Sec. S1.2 of the supplement.

5.3. Analysis

We show in Fig. 3 an in-depth analysis on how a TR scheduler works during QAT. We can see from Fig. 3a that the running TR K^t roughly follows the target TR R^t , indicating that we can control the average effective step size of quantized weights (the red curve in Fig. 1c) by scheduling the target TR. This is possible because the TALR U^t is adjusted adaptively to match the running TR K^t with the

target one R^t (Fig. 3b). We can see that the TALR U^t increases initially, since the running TR K^t is much smaller than the target TR R^t . The TALR U^t then decreases gradually to reduce the number of transitions, following the target TR R^t . Note that the TALR U^t approaches zero rapidly near the 50K-th iteration. To figure out the reason, we show in Figs. 3c and 3d distributions of normalized latent weights and their average distances to the nearest transition points, respectively. We can observe in Fig. 3c that latent weights tend to be concentrated near the transition points of a quantizer as the training progresses, similar to the case in Sec. 4.1 using a user-defined LR. This implies that transitions occur more frequently in later training iterations if we do not properly reduce the degree of parameter change for latent weights. In particular, we can see in Fig. 3d the average distances between the normalized latent weights and the nearest transition points are relatively small after the 50K-th iteration. Under such circumstance, the TALR should become much smaller in order to reduce the running TR, as in the sharp decline around the 50K-th iteration. We can thus conclude that our approach adjusts the TALR by considering the distribution of the latent weights implicitly.

6. Conclusion

We have discussed the problem of a conventional optimization method using a LR in QAT. To overcome this, we have presented a TR scheduling technique specialized for general QAT, which controls the number of transitions in quantized weights explicitly. We have shown that the optimizers coupled with our TR scheduling technique outperform the plain ones using a LR by significant margins under various QAT settings. We have also verified that our approach enables stable training with different types of optimizers and schedulers, which is generally difficult in the conventional optimization methods, indicating that the TR scheduling technique offers more diverse training options for QAT. We expect that our method would be adopted to boost the performance of other quantized models, bringing a significant advance in the field of network quantization.

Acknowledgements

This work was supported in part by NRF and IITP grants funded by the Korea government (MSIT) (No. 2023R1A2C2004306, No.RS-2022-00143524, Development of Fundamental Technology and Integrated Solution for Next-Generation Automatic Artificial Intelligence System, RS-2021-II212068, Artificial Intelligence Innovation Hub) and the Yonsei Signature Research Cluster Program of 2025 (2025-22-0013).

References

- [1] Yu Bai, Yu-Xiang Wang, and Edo Liberty. ProxQuant: Quantized neural networks via proximal operators. In *Int. Conf. Learn. Represent.*, 2019. [1](#), [3](#)
- [2] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv*, 2013. [2](#), [4](#), [17](#)
- [3] Yash Bhalgat, Jinwon Lee, Markus Nagel, Tijmen Blankevoort, and Nojun Kwak. LSQ+: Improving low-bit quantization through learnable offsets and better initialization. In *IEEE Conf. Comput. Vis. Pattern Recog. Workshops*, pages 696–697, 2020. [6](#), [11](#), [12](#)
- [4] Zhaowei Cai, Xiaodong He, Jian Sun, and Nuno Vasconcelos. Deep learning with low precision by half-wave gaussian quantization. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 5918–5926, 2017. [1](#), [2](#)
- [5] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. PACT: Parameterized clipping activation for quantized neural networks. *arXiv*, 2018. [1](#), [2](#), [4](#), [11](#), [12](#)
- [6] Alexandre Défossez, Yossi Adi, and Gabriel Synnaeve. Differentiable model compression via pseudo quantization noise. *TMLR*, 2022. [3](#)
- [7] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. ImageNet: A large-scale hierarchical image database. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 248–255, 2009. [2](#), [6](#), [7](#), [11](#), [12](#)
- [8] Timothy Dozat. Incorporating Nesterov momentum into Adam. In *Int. Conf. Learn. Represent. Workshop*, 2016. [15](#)
- [9] John Duchi, Elad Hazan, and Yoram Singer. Adaptive sub-gradient methods for online learning and stochastic optimization. *J. Mach. Learn. Res.*, 12(7), 2011. [3](#), [15](#)
- [10] Steven K Esser, Jeffrey L McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S Modha. Learned step size quantization. In *Int. Conf. Learn. Represent.*, 2020. [1](#), [2](#), [4](#), [6](#), [11](#), [12](#), [15](#), [17](#), [19](#)
- [11] Ruihao Gong, Xianglong Liu, Shenghu Jiang, Tianxiang Li, Peng Hu, Jiazhen Lin, Fengwei Yu, and Junjie Yan. Differentiable soft quantization: Bridging full-precision and low-bit neural networks. In *Int. Conf. Comput. Vis.*, pages 4852–4861, 2019. [1](#), [2](#), [11](#), [12](#)
- [12] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 770–778, 2016. [1](#), [2](#), [4](#), [6](#), [7](#), [8](#), [11](#), [12](#), [13](#), [14](#), [15](#), [16](#), [17](#)
- [13] Koen Helwegen, James Widdicombe, Lukas Geiger, Zechun Liu, Kwang-Ting Cheng, and Roeland Nusselder. Latent weights do not exist: Rethinking binarized neural network optimization. In *Adv. Neural Inform. Process. Syst.*, pages 7533–7544, 2019. [3](#)
- [14] Byeongho Heo, Sanghyuk Chun, Seong Joon Oh, Dongyoon Han, Sangdoo Yun, Gyuwan Kim, Youngjung Uh, and Jung-Woo Ha. AdamP: Slowing down the slowdown for momentum optimizers on scale-invariant weights. In *Int. Conf. Learn. Represent.*, 2021. [4](#), [5](#), [6](#)
- [15] Elad Hoffer, Ron Banner, Itay Golan, and Daniel Soudry. Norm matters: efficient and accurate normalization schemes in deep networks. In *Adv. Neural Inform. Process. Syst.*, 2018. [4](#), [5](#), [6](#)
- [16] Andrew G Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. MobileNets: Efficient convolutional neural networks for mobile vision applications. *arXiv*, 2017. [6](#)
- [17] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 7132–7141, 2018. [1](#)
- [18] Gao Huang, Yixuan Li, Geoff Pleiss, Zhuang Liu, John E Hopcroft, and Kilian Q Weinberger. Snapshot ensembles: Train 1, get m for free. In *Int. Conf. Learn. Represent.*, 2017. [1](#), [3](#), [4](#), [16](#)
- [19] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 4700–4708, 2017. [1](#)
- [20] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Int. Conf. Mach. Learn.*, pages 448–456, 2015. [4](#), [5](#), [6](#), [11](#)
- [21] Sangil Jung, Changyong Son, Seohyung Lee, Jinwoo Son, Jae-Joon Han, Youngjun Kwak, Sung Ju Hwang, and Changkyu Choi. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 4350–4359, 2019. [1](#), [2](#), [4](#), [11](#), [12](#), [17](#), [19](#)
- [22] Dohyung Kim, Junghyup Lee, and Bumsu Ham. Distance-aware quantization. In *Int. Conf. Comput. Vis.*, pages 5271–5280, 2021. [1](#), [2](#), [15](#)
- [23] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Int. Conf. Learn. Represent.*, 2014. [1](#), [2](#), [3](#), [5](#), [7](#), [12](#), [14](#), [15](#)
- [24] Bobby Kleinberg, Yuanzhi Li, and Yang Yuan. An alternative view: When does SGD escape local minima? In *Int. Conf. Mach. Learn.*, pages 2698–2707, 2018. [1](#), [3](#), [4](#), [16](#)
- [25] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Technical report, 2009. [2](#), [4](#), [7](#), [8](#), [14](#), [15](#), [16](#)
- [26] Junghyup Lee, Dohyung Kim, and Bumsu Ham. Network quantization with element-wise gradient scaling. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 6448–6457, 2021. [1](#), [3](#), [5](#), [6](#), [11](#), [12](#), [15](#), [17](#), [19](#)
- [27] Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. *arXiv*, 2016. [1](#), [2](#)

- [28] Yuanzhi Li, Colin Wei, and Tengyu Ma. Towards explaining the regularization effect of initial large learning rate in training neural networks. In *Adv. Neural Inform. Process. Syst.*, 2019. 3
- [29] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Eur. Conf. Comput. Vis.*, pages 740–755, 2014. 2, 7, 8, 11, 13
- [30] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Int. Conf. Comput. Vis.*, pages 2980–2988, 2017. 2, 7, 8, 11, 13, 17
- [31] Zechun Liu, Baoyuan Wu, Wenhan Luo, Xin Yang, Wei Liu, and Kwang-Ting Cheng. Bi-Real Net: Enhancing the performance of 1-bit cnns with improved representational capability and advanced training algorithm. In *Eur. Conf. Comput. Vis.*, pages 722–737, 2018. 6
- [32] Zechun Liu, Zhiqiang Shen, Marios Savvides, and Kwang-Ting Cheng. ReActNet: Towards precise binary neural network with generalized activation functions. In *Eur. Conf. Comput. Vis.*, pages 143–159, 2020. 1, 2, 6, 7, 12, 14, 15, 17
- [33] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *Int. Conf. Learn. Represent.*, 2017. 3, 8, 14, 15, 17
- [34] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *Int. Conf. Learn. Represent.*, 2019. 2, 7, 15
- [35] Markus Nagel, Marios Fournarakis, Yelysei Bondarenko, and Tijmen Blankevoort. Overcoming oscillations in quantization-aware training. In *Int. Conf. Mach. Learn.*, 2022. 3, 5, 11, 12, 16, 17
- [36] Eunhyeok Park and Sungjoo Yoo. PROFIT: A novel training method for sub-4-bit MobileNet models. In *Eur. Conf. Comput. Vis.*, pages 430–446, 2020. 5, 11, 16
- [37] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *Eur. Conf. Comput. Vis.*, pages 525–542, 2016. 1, 2, 17
- [38] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. MobileNetV2: Inverted residuals and linear bottlenecks. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 4510–4520, 2018. 2, 7, 12
- [39] Pedro Savarese, Xin Yuan, Yanjing Li, and Michael Maire. Not all bits have equal value: Heterogeneous precisions via trainable noise. In *NeurIPS*, 2022. 3
- [40] Juncheol Shin, Junhyuk So, Sein Park, Seungyeop Kang, Sungjoo Yoo, and Eunhyeok Park. Nipq: Noise proxy-based integrated pseudo-quantization. In *CVPR*, 2023. 3
- [41] Cuauhtemoc Daniel Suarez-Ramirez, Miguel Gonzalez-Mendoza, Leonardo Chang, Gilberto Ochoa-Ruiz, and Mario Alberto Duran-Vega. A bop and beyond: A second order optimizer for binarized neural networks. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 1273–1281, 2021. 3
- [42] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 2818–2826, 2016. 1
- [43] T. Tieleman and G. Hinton. Lecture 6.5 - RMSProp: Divide the gradient by a running average of its recent magnitude, COURSERA: Neural Networks for Machine Learning. Technical report, 2012. 3, 15
- [44] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *ICML*, 2021. 2, 7
- [45] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu, and Kaiming He. Aggregated residual transformations for deep neural networks. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 1492–1500, 2017. 1
- [46] Kohei Yamamoto. Learnable companding quantization for accurate low-bit neural networks. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 5029–5038, 2021. 1, 2, 7, 15, 17
- [47] Jiwei Yang, Xu Shen, Jun Xing, Xinmei Tian, Houqiang Li, Bing Deng, Jianqiang Huang, and Xian-sheng Hua. Quantization networks. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 7308–7316, 2019. 1, 2, 11, 12
- [48] Dongqing Zhang, Jiaolong Yang, Dongqiangzi Ye, and Gang Hua. LQ-Nets: Learned quantization for highly accurate and compact deep neural networks. In *Eur. Conf. Comput. Vis.*, pages 365–382, 2018. 1, 2
- [49] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou. DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv*, 2016. 1, 2, 4, 17
- [50] Bohan Zhuang, Lingqiao Liu, Minghui Tan, Chunhua Shen, and Ian Reid. Training quantized neural networks with a full-precision auxiliary module. In *IEEE Conf. Comput. Vis. Pattern Recog.*, pages 1488–1497, 2020. 7

Scheduling Weight Transitions for Quantization-Aware Training

Supplementary Material

In the supplement, we give additional results in image classification and object detection (Sec. S1). We then provide in-depth analyses (Sec. S2) and discussions on our method (Sec. S3). Finally, we present the implementation details of our method (Sec. S4) and comprehensive description of a quantizer (Sec. S5). We summarize in Algorithm 1 an overall process of our method.

S1. Results

S1.1. Image classification

Comparison to the state of the art We present in Table S1 a quantitative comparison of our approach and state-of-the-art methods for QAT [3, 5, 10, 11, 21, 26, 35, 47] on ImageNet [7], in terms of a top-1 validation accuracy. For a fair comparison, we perform comparisons with the methods using uniform quantization schemes and a vanilla architecture. We can see from the table that our method (*i.e.*, using either SGDT or AdamT) achieves state-of-the-art performance for ResNet-18 under the 1/1 and 2/2 bit-width settings, and shows competitive performance for the 3/3 and 4/4 bit-width settings. In terms of performance gains or drops compared to the full-precision models, our results are better than or on a par with the state of the art. For MobileNetV2, our method also sets a new state of the art, except for the 3/3 bit-width setting. The work of [35] alleviates the oscillation problem in QAT by freezing latent weights or adding a regularization term. It adopts architecture-specific and bit-specific hyperparameters, such as a LR, a weight decay, and a threshold for weight freezing. This is because freezing weights and adding the regularizer could potentially degrade trainability and disturb the training process, making the QAT process sensitive to hyperparameters. In contrast, our method is more practical as it performs consistently well under the various bit-width settings with the same set of hyperparameters. For completeness, we also try to tune a TR factor λ specific to the 3/3-bit setting of MobileNetV2, and improve the performance with the TR factor λ of $2e-3$.

Other QAT methods mostly focus on designing a quantizer in a forward step (*e.g.*, PACT [5], QIL [21], LSQ [10], LSQ+ [3]) or addressing a gradient mismatch problem in a backward step (*e.g.*, QNet [47], DSQ [11], EWGS [26]). Our approach is orthogonal to these methods in that we focus on an optimization step in QAT. Namely, we propose to schedule a target TR instead of a LR itself, and introduce a novel TALR to update latent weights. Compared to other methods that require computationally expensive operations for differentiable quantizers (*e.g.*, QNet[47], DSQ [11]) or

Algorithm 1 Optimization process using a TR scheduler.

Require: the number of iterations T ; a target TR R^t ; a momentum constant m .

- 1: **while** $t < T$ **do**
 - 2: Compute a current TR k^t [Eq. (5)]:

$$k^t \leftarrow \frac{\sum_{i=1}^N \mathbb{1}[w_d^t(i) \neq w_d^{t-1}(i)]}{N}$$
 - 3: Estimate a running TR K^t [Eq. (10)]:

$$K^t \leftarrow mK^{t-1} + (1 - m)k^t$$
 - 4: Adjust a TALR U^t [Eq. (11)]:

$$U^t \leftarrow \max(0, U^{t-1} + \eta(R^t - K^t))$$
 - 5: Update latent weights $\mathbf{w}^t$ with a gradient term $\mathbf{g}^t$ and the TALR U^t [Eq. (12)]:

$$\mathbf{w}^{t+1} \leftarrow \mathbf{w}^t - U^t \mathbf{g}^t$$
 - 6: **end while**
-

approximating a hessian trace (*e.g.*, EWGS [26]), ours is relatively simple and efficient, since it adds few comparison and scalar operations only in the QAT process.

Quantitative results. We show in Fig. S1 training curves on ImageNet for MobileNetV2 and ReActNet-18 using 4-bit and binary weights/activations, respectively. We can see from Figs. S1a and S1b that the optimizer coupled with our TR scheduling technique (*i.e.*, AdamT) can control the average effective step size of quantized weights roughly using a target TR. On the other hand, the plain optimizer adopting a user-defined LR (*i.e.*, Adam) fails to control it by scheduling the LR. For example, the average effective step sizes for quantized weights in MobileNetV2 (*i.e.*, the blue curve in the first row of Fig. S1b) do not approach zero at the end of training. Such large changes in the quantized weights can lead to unstable batch normalization statistics [20], resulting in noisy and degraded test time performance [35, 36] (*i.e.*, the blue curve in the first row of Fig. S1d). On the contrary, the TR scheduling technique enables better training in terms of the convergence rate and performance, which can be verified by the training losses and validation accuracy in Figs. S1c and S1d, respectively.

S1.2. Object detection

Quantitative results. We compare in Table S2 the quantization performance of detection models in terms of an average precision (AP) on the validation split of MS COCO [29]. We train RetinaNet [30] with the ResNet-18/34 [12] backbones using either SGD or SGDT on the training split of MS COCO. From Table 4 of the main paper and Table S2, we can see that using a TR scheduling technique improves the performance of quantized models

Table S1. Quantitative comparison of our method and the state of the art on ImageNet [7] in terms of a top-1 validation accuracy. The numbers in parentheses indicate the performance gains or drops compared to the full-precision (32/32) models. All numbers are taken from corresponding works except for LSQ [10], where the results are reproduced in [3] using a vanilla structure of ResNet [12].

Architecture	Methods	Optimizer	Weight/activation bit-widths				
			1/1	2/2	3/3	4/4	32/32
ResNet-18	PACT [5]	SGD	-	64.4 (−5.8)	68.1 (−2.1)	69.2 (−1.0)	70.2
	QIL [21]	SGD	-	65.7 (−4.5)	69.2 (−1.0)	70.1 (−0.1)	70.2
	QNet [47]	SGD	53.6 (−16.7)	-	-	-	70.3
	DSQ [11]	-	-	65.2 (−4.7)	68.7 (−1.2)	69.6 (−0.3)	69.9
	LSQ [10]	SGD	-	66.7 (−3.4)	<u>69.4</u> (−0.7)	<u>70.7</u> (+0.6)	70.1
	LSQ+ [3]	SGD	-	66.8 (−3.3)	69.3 (−0.8)	70.8 (+0.7)	70.1
	EWGS [26]	SGD	55.3 (−14.6)	<u>67.0</u> (−2.9)	69.7 (−0.2)	70.6 (+0.7)	69.9
	Ours	SGDT	<u>55.8</u> (−14.1)	66.9 (−3.0)	69.7 (−0.2)	70.6 (+0.7)	69.9
	Ours	AdamT	56.3 (−13.6)	67.2 (−2.7)	69.7 (−0.2)	70.4 (+0.5)	69.9
MobileNetV2	DSQ [11]	-	-	-	-	64.8 (−7.1)	71.9
	EWGS [26]	SGD	-	-	-	70.3 (−1.6)	71.9
	Nagel <i>et al.</i> [35]	SGD	-	-	<u>67.6</u> (−4.1)	<u>70.6</u> (−1.1)	71.7
	Ours	SGDT	-	<u>53.6</u> (−18.3)	67.0 (−4.9)	70.5 (−1.4)	71.9
	Ours	AdamT	-	53.8 (−18.1)	67.3 (−4.6)	70.8 (−1.1)	71.9
	Ours ($\lambda=2e-3$)	AdamT	-	-	67.8 (−4.1)	-	71.9

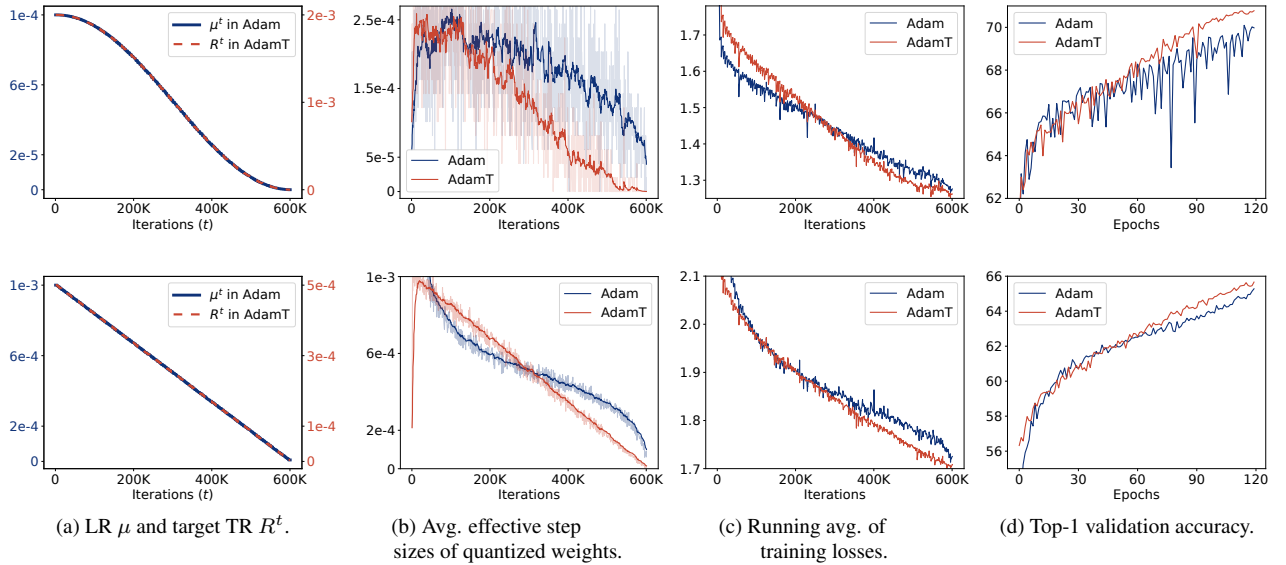


Figure S1. Training curves for quantized models using Adam [23] and AdamT on ImageNet [7]. The results in the first and second rows are obtained with MobileNetV2 [38] and ReActNet-18 [32] using 4-bit and binary weights/activations, respectively. For the visualizations in (b), we monitor the average effective step sizes of quantized weights in the 19th and 17th layers of MobileNetV2 and ReActNet-18, respectively. (Best viewed in color.)

in terms of AP, compared to the plain optimization method, across different bit-width and backbone settings. This confirms once again the effectiveness and generalization ability of our approach.

Qualitative results. We compare in Fig. S2 qualitative results of baseline (SGD) and our (SGDT) models, us-

ing RetinaNet with the ResNet-50 backbone, with 4-bit weights/activations. The baseline model trained with SGD provides incorrect bounding boxes in the presence of overlapped/adjacent instances (*e.g.*, the cow and teddy bear in the first and second examples, respectively), misclassifies object classes (*e.g.*, the food container in the third example), and fails to detect objects (*e.g.*, the bicycle in the last

Table S2. Quantitative comparison of quantized models on object detection. We train RetinaNet [30] on the training split of MS COCO [29] with different backbone networks and quantization bit-widths, using either the plain optimization method (SGD) or ours (SGDT). We report the average precision (AP) on the validation split.

Backbone	W/A	Optimizer	AP	AP ₅₀	AP ₇₅	AP _S	AP _M	AP _L
ResNet-34	FP	SGD	37.70	57.32	40.71	22.35	41.50	48.93
		SGDT	37.70	57.32	40.71	22.35	41.50	48.93
	4/4	SGD	37.99	57.29	40.34	22.96	41.16	49.32
		SGDT	38.14	57.39	40.93	22.05	41.70	49.79
	3/3	SGD	37.32	56.61	39.95	22.22	40.54	49.16
		SGDT	37.64	56.78	40.28	21.81	40.42	49.89
ResNet-18	FP	SGD	34.06	53.15	36.17	19.66	36.48	44.45
		SGDT	34.06	53.15	36.17	19.66	36.48	44.45
	4/4	SGD	35.09	54.05	37.40	20.34	37.62	46.69
		SGDT	35.30	54.46	37.59	19.92	37.61	46.83
	3/3	SGD	34.32	53.09	36.44	19.34	36.46	45.33
		SGDT	34.72	53.63	36.84	19.91	37.21	45.64

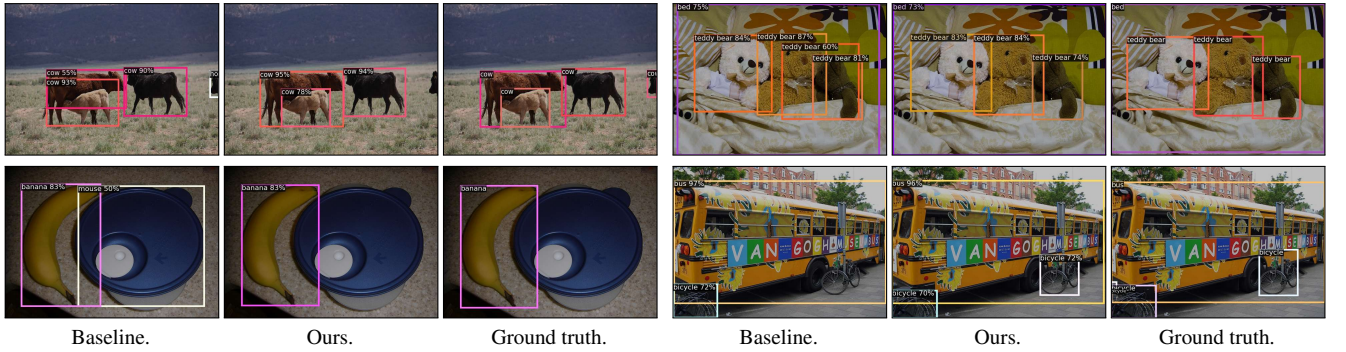


Figure S2. Qualitative comparison for object detection on MS COCO [29] using RetinaNet [30] with the ResNet-50 [12] backbone, where both weights and activations are quantized into 4-bit. The results of baseline and ours are obtained from the models trained with SGD and SGDT, respectively. (Best viewed in color.)

example). In contrast, the model trained with our approach offers accurate bounding boxes, and classifies object classes successfully.

S2. Analysis

S2.1. Analysis on TR scheduling

We show in Fig. S3 an in-depth analysis on how a TR scheduler works during QAT. We can see from Fig. S3a that the running TR K^t roughly follows the target TR R^t , indicating that we can control the average effective step size of quantized weights (the red curve in Fig. 1c of the main paper) by scheduling the target TR. This is possible because the TALR U^t is adjusted adaptively to match the running TR K^t with the target one R^t (Fig. S3b). We can see that the TALR U^t increases initially, since the running TR K^t is much smaller than the target TR R^t . The TALR U^t then decreases gradually to reduce the number of transitions, following the target TR R^t . Note that the TALR U^t approaches

zero rapidly near the 50K-th iteration. To figure out the reason, we show in Figs. S3c and S3d distributions of normalized latent weights and their average distances to the nearest transition points, respectively. We can observe in Fig. S3c that latent weights tend to be concentrated near the transition points of a quantizer as the training progresses, similar to the case in Sec. 4.1 in the main paper using a user-defined LR. This implies that transitions occur more frequently in later training iterations if we do not properly reduce the degree of parameter change for latent weights. In particular, we can see in Fig. S3d the average distances between the normalized latent weights and the nearest transition points are relatively small after the 50K-th iteration. Under such circumstance, the TALR should become much smaller in order to reduce the running TR, as in the sharp decline around the 50K-th iteration. We can thus conclude that our approach adjusts the TALR by considering the distribution of the latent weights implicitly.

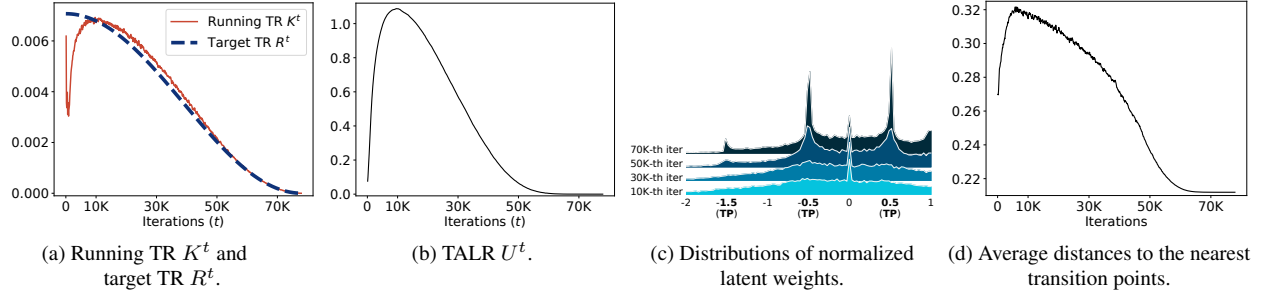


Figure S3. Analysis on TR scheduling. We train ResNet-20 [12] on CIFAR-100 [25] using SGDT, where we quantize both weights and activations with 2-bit representations. We visualize distributions of normalized latent weights in the 16th layer in (c), and average distances between normalized latent weights and the nearest transition points in (d). The transition points in (c) are denoted by TPs in the x-axis. The top-1 test accuracy and average effective step sizes of quantized weights are shown by the red curves in Figs. 1d and 1c of the main paper, respectively. (Best viewed in color.)

Table S3. Quantitative comparison of optimization methods for QAT using a step decay scheduler on CIFAR-100/10 [25]. Both LR and target TRs for SGD and SGDT are divided by 5 after every 50 and 100 epochs for ReActNet-18 [32] and ResNet-20 [12], respectively. Numbers in parentheses indicate accuracy drops compared to the models in Table 2 of the main paper trained with a cosine scheduler.

Optimizer	CIFAR-100			CIFAR-10		
	ReActNet-18 (W32A1: 69.6)	ResNet-20 (FP: 65.1)		ReActNet-18 (W32A1: 91.3)	ResNet-20 (FP: 91.1)	
	1/1	1/1	2/2	1/1	1/1	2/2
SGD	69.0 (−0.7)	45.8 (−9.1)	61.3 (−2.8)	90.9 (−0.0)	82.9 (−2.3)	89.8 (−0.4)
SGDT	71.9 (−0.3)	55.3 (−0.5)	64.9 (−0.6)	93.0 (−0.0)	85.2 (−0.4)	90.3 (−0.4)
Adam	68.1 (−1.4)	49.9 (−4.9)	60.9 (−2.4)	82.5 (−7.9)	82.4 (−2.4)	89.6 (−0.6)
AdamT	71.7 (−0.1)	54.4 (−1.5)	64.6 (−0.6)	92.9 (−0.0)	85.3 (−0.4)	90.3 (−0.8)

S2.2. Comparison using a step decay scheduler

We provide in Table S3 a quantitative comparison between plain optimization methods (SGD and Adam [23]) and ours (SGDT and AdamT) using a step scheduler. We use the same training setting in Table 2 of the main paper, while replacing a cosine scheduler [33] with the step scheduler. Both LR and target TRs are divided by 5 after every 50 and 100 epochs for ReActNet-18 [32] and ResNet-20 [12], respectively. We can see from this table that the plain optimizers using the step scheduler suffer from significant accuracy drops, compared to the cases for the cosine scheduler. The main reason is that it is difficult to control the average effective step size of quantized weights using a user-defined LR, especially when the LR is decayed by the step scheduler. On the other hand, the optimizers using our TR scheduling technique are relatively robust to the step scheduler, showing less performance drops than the plain ones for most cases. We can also observe that the performance degrades more severely for the models using the ResNet-20 [12] architecture, where the number of parameters is much less than that of the ReActNet-18 [32] architecture (276K for ResNet-20 vs. 11,235K for ReActNet-

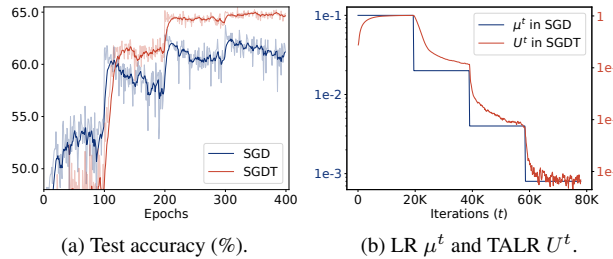


Figure S4. Comparison between SGD and SGDT using a step scheduler. We train ResNet-20 [12] with 2-bit weights and activations on CIFAR-100 [25]. Both the LR for SGD and target TR for SGDT are divided by 5 after every 100 epochs. For comparison, we show in (b) the LR μ^t in SGD and TALR U^t in SGDT, where both are used for updating latent weights in Eqs. (4) and (12) of the main paper. (Best viewed in color.)

18). This indicates that training a small quantized model with the step scheduler is more challenging, possibly because the scheduler can cause large parameter changes even at the end of the training.

To further analyze the difference between the LR and

Table S4. Quantitative comparison of different TR factors λ (*i.e.*, initial target TRs). We quantize ResNet-20 [12] using SGDT with 2-bit weights/activations, and report a top-1 test accuracy on CIFAR-100 [25].

TR factor λ	1e-3	2e-3	3e-3	4e-3	5e-3	6e-3	7e-3	8e-3	9e-3	1e-2
Test accuracy	62.5	64.2	64.3	65.3	65.5	65.1	63.1	63.6	63.6	64.0

TR scheduling techniques with the step scheduler, we compare in Fig. S4 the LR in SGD and TALR in SGDT during training, with corresponding test accuracies. We can observe from Fig. S4a that SGDT provides better results in terms of both convergence rate and accuracy. As discussed in Sec. 4.1 of the main paper, latent weights approach transition points progressively during QAT, and thus it is difficult to adjust the number of transitions explicitly using a user-defined LR. This could be particularly problematic when the LR is fixed for a number of iterations. Recent QAT methods [10, 22, 26, 32, 46] circumvent this issue by decaying a LR to zero gradually, *e.g.*, using the cosine annealing technique [33], but this does not fully address the problem. On the other hand, our method is relatively robust to the use of the step scheduler. Since we optimize latent weights using a TALR, we can control the degree of parameter changes for quantized weights by scheduling the target TR in our method. In contrast to the LR, the TALR decreases gradually even when the scheduler does not alter the target TR (Fig. S4b), confirming once more the fact that the TALR is adjusted adaptively considering the distribution of latent weights. This result demonstrates the robustness and generalization ability of our approach on various types of schedulers.

S2.3. Analysis on initial target TRs

To better understand how an initial target TR influences the quantization performance, we present in Table S4 a quantitative comparison of different TR factors λ adjusting the initial TRs. We train ResNet-20 [12] models on CIFAR-100 [25] using 2-bit weights and activations with SGDT and report top-1 test accuracies. We can observe that the TR factors within a wide range of intervals (*i.e.*, [4e-3, 6e-3]) provide satisfactory performance, outperforming the LR-based optimization strategy (*i.e.*, SGD in Table 2 of the main paper) by significant margins (1.0~1.4). Similar to the LR, extremely large or small TR factors degrade the quantization performance, which could make the training unstable. Therefore, setting an appropriate value for the TR factor is crucial in the TR scheduling technique, analogous to determining an initial LR to train full-precision models.

Table S5. Quantitative comparison for plain optimization methods and our TR scheduling technique with various optimizers. We report a top-1 test accuracy on CIFAR-100 [25] using ResNet-20 [12] with 2-bit weights/activations.

	SGD	Adam	NAdam	Adamax	AdamW	RMSProp	Adagrad
Plain	64.1	63.3	63.3	62.4	63.8	64.6	54.2
Ours	65.5	65.2	65.1	64.7	66.2	65.1	61.1

Table S6. Quantitative comparison of quantized models trained with SGDT using different final target TRs. We report mean and standard deviation of top-1 test accuracies on CIFAR-100 [25] over five random runs, obtained with ResNet-20 [12] using 2-bit weights and activations.

Final target TR	0	1e-5	1e-4	1e-3
Final target average effective step size	0	5e-6	5e-5	5e-4
Test accuracy	65.61 (± 0.21)	65.34 (± 0.26)	64.63 (± 0.59)	62.12 (± 0.70)

Table S7. Training time comparison on ImageNet with 4 A5000 GPUs, in terms of GPU hours. We quantize ResNet-18 except for AdamW/-T, where we quantize DeiT-T.

	SGD	Adam	AdamW
Plain	174	175	477
Ours	176	178	492

S2.4. Comparison with different optimizers

To verify the generalization ability of our TR scheduling technique to other optimizers, we compare in Table S5 the quantization performance between plain optimization methods and ours with various optimizers, including SGD, Adam [23], NAdam [8], Adamax [23], AdamW [34], RMSProp [43], and Adagrad [9]. Specifically, we train ResNet-20 on CIFAR-100 [25] with 2-bit weights and activations for comparison. For the optimizers other than SGD, we exploit the same hyperparameters as the ones for Adam in Table 2 of the main paper, while we change the weight decay to 1e-2 for AdamW. Note that AdamW decouples the weight decay from a parameter update step, suggesting that it requires a different hyperparameter for the weight decay. For RMSProp, we use the momentum gradient with a momentum value of 0.9, which is analogous to SGD. From the table, we can clearly see that the TR scheduling technique improves the quantization performance consistently across all optimizers. Our method achieves 0.5%~6.9% gains over the plain optimization methods, which demonstrates the effectiveness of our method and its generalization ability to various optimizers. In particular, AdamW coupled with our TR scheduling technique shows the best performance, indicating that we could further improve the quantization

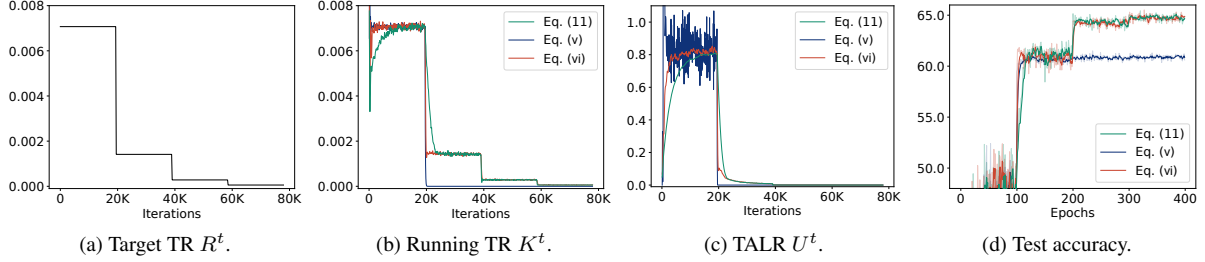


Figure S5. Comparison of different update schemes for a TALR. We train ResNet-20 [12] on CIFAR-100 [25] with 2-bit weights and activations, while adjusting the TALR based on different methods in Eqs. (11), (S1) and (S2). (Best viewed in color.)

performance by carefully selecting an optimizer and corresponding hyperparameters.

S2.5. Training time comparison

We compare in Table S7 training time of optimization methods. Our method takes training time, similar to plain optimization methods, demonstrating its efficiency. The marginal overheads of ours is due to the computation of the TR and the adjustment of the TALR, both of which involve lightweight operations such as element-wise comparisons and scalar updates, which are computationally inexpensive compared to the overall training process.

S3. Discussion

S3.1. Importance of reducing average effective step size

Conventional optimization methods for full-precision models typically decay a LR to reduce the degree of parameter changes (*i.e.*, average effective step sizes). This prevents full-precision weights from overshooting from a local optimum, and improves the convergence of the model [18, 24]. Similarly, when training a quantized model, small average effective step sizes for quantized weights are preferred in later training iterations. Large changes in the quantized weights at the end of training could disturb convergence and degrade the quantization performance [35, 36], as shown in Figs. 1, 2, and S1. To further support this, we perform an experiment on the influence of the average effective step size of quantized weights on the quantization performance at the end of training. To this end, we train ResNet-20 [12] on CIFAR-100 [25] with 2-bit weights and activations using SGDT for different final target TRs. We report in Table S6 the mean and standard deviation of top-1 test accuracies over five random runs for each model. We can clearly see that the smaller average effective step sizes at the end of training provide better performance with less deviation, suggesting that the large step sizes in later training iterations disturb convergence. These results demonstrate once more the importance of the TR scheduling technique,

since it is difficult to control the average effective step size of quantized weights, especially in later training iterations, with conventional LR scheduling in QAT.

S3.2. Comparison of update schemes for TALR

We adjust a TALR adaptively to match a current running TR with a target one. To this end, the update rule for the TALR should meet the following criteria: (1) If the running TR K^t is smaller than the target one R^t , the TALR U^t should increase to allow more latent weights to pass transition points. (2) If the running TR K^t is larger than the target one R^t , the TALR U^t should decrease to reduce the number of transitions. (3) If the current running TR K^t matches the target one R^t , the TALR U^t should remain unchanged. We currently adjust the TALR in an additive manner in Eq. (11) of the main paper, similar to the weight update in gradient-based optimizers. Here, we consider two alternative variants. First, we update the TALR in a multiplicative manner as follows:

$$U^t = U^{t-1} \frac{R^t}{K^t}. \quad (\text{S1})$$

While the multiplication allows fast adaptation for the TALR, it might be sensitive to outliers from the running TR K^t or drastic changes in the target TR R^t . To stabilize the update, we adjust the TALR in a similar way to Eq (S1), but with momentum, as follows:

$$U^t = m' U^{t-1} + (1 - m') U^{t-1} \frac{R^t}{K^t}, \quad (\text{S2})$$

where m' is a momentum hyperparameter, which is set to the same value as m in Eq. (10) of the main paper for simplicity. To compare the update schemes, we analyze in Fig. S5 the QAT process of ResNet-20 [12] on CIFAR-100 [25]. We can see from Figs. S5a and S5b that the running TRs K^t closely follow a target one R^t with the update schemes of Eqs. (11) and (S2). The scheme in Eq. (S1) fails to follow the target TR, resulting in the worst performance (Fig. S5d). The multiplicative update in Eq. (S1) enables fast adaptation, but it is unstable and particularly problematic when the target TR R^t changes abruptly by a

step scheduler. The momentum update in Eq. (S2), on the other hand, offers a good compromise between stability and adaptation speed, achieving the quantization performance comparable to the update scheme of Eq. (11) of the main paper. Although we adopt the scheme of Eq. (11) in our TR scheduling technique for conciseness, Fig. S5 suggests that how to adjust the TALR is not unique, and it can be updated in various ways.

S3.3. Average effective step size

The TR scheduling technique adjusts a TR of quantized weights (*i.e.*, Eq. (5) of the main paper) explicitly, which in turn controls the degree of parameter changes for the quantized weights. As a variant of our method, we could instead formulate k^t in Eq. (5) of the main paper with the average effective step size of the quantized weights as follows:

$$k^t = \frac{\sum_{i=1}^N |w_q^t(i) - w_q^{t-1}(i)|}{N}. \quad (\text{S3})$$

In this way, we could control the average effective step size of the quantized weights directly during training, similar to the TR in our method. Note that the effects of our method and its variant are nearly the same, if the quantized weights transit between adjacent discrete levels as discussed in Eq. (9) of the main paper. This is always true for binary weights, or usually happens especially when the parameter updates for latent weights are small. When adopting the variant in Eq. (S3), however, it could be difficult to set a hyperparameter (*i.e.*, an initial target value for the average effective step size) due to different scales in quantized weights. For example, several quantization methods [10, 37] provide quantized weights whose min-max ranges are different depending on layers in a model. In this case, we should use different search ranges for individual layers, to set initial target values, since the scales of average effective step sizes could vary in different layers. It is however computationally demanding to find an optimal initial target value for each layer separately. Accordingly, we focus on scheduling the TR rather than the average effective step size, which is more easy to use and generalizable to various quantization schemes.

S4. Implementation Details

S4.1. Experimental settings

Following the experimental protocol in [21, 26, 32, 46, 49], we initialize network weights from pretrained full-precision models for MobileNetV2, ResNets, DeiT_s and RetinaNet. Similarly, the weights in ReActNet-18 are initialized using the pretrained activation-only binarized model. We do not quantize the first and last layers. We use either plain optimizers using a LR (SGD, Adam, and AdamW), or their variants using our TR scheduling technique (SGDT,

AdamT, and AdamWT), whose gradient terms are the same as SGD, Adam, and AdamW, respectively. Note that we use a TR scheduler to the latent weights only (*i.e.*, full-precision weights coupled with quantizers). Other parameters that are not quantized (*e.g.*, the weights in the first and last layers) are updated with the plain optimizers using a LR. We adopt the cosine scheduler [33] for both the LR and the target TR, decaying them to zero gradually. For ReActNet-18 on ImageNet, we use the linear scheduler, following the training scheme in [32]. We exploit STE [2] to propagate gradients in discretization functions. To set an initial value of the learnable scale parameter s in Eq. (13) of the main paper, we follow the initialization technique in [26]. For an initial TALR, we use the same value as an initial LR for full-precision parameters. We also set η in Eq. (11) of the main paper to the same value as an initial TALR (*i.e.*, $\eta = U^0$), adjusting the TALR according to its initial value, *e.g.*, updating the TALR more finely when the initial value is small. Note that the number of transition points increases as bit-widths of weights get larger, suggesting that a larger target TR is desirable for larger bit-widths. Accordingly, motivated by [10] that uses different gradient scales according to bit-widths, we set the initial target TR to $\lambda\sqrt{b_w}$, where λ is a TR factor, a hyperparameter to set, and b_w is the bit-width of weights.

We summarize in Table S8 hyperparameter settings for our experiments. If available, we follow the previous works [10, 26, 30, 32, 35] to set hyperparameters (*e.g.*, number of training epochs/iterations, batch size, LR, weight decay, and decay scheduler). Different from the original work of RetinaNet [30], we do not freeze the ResNet [12] backbones to optimize quantized weights via QAT, and adopt the cosine decay scheduler [33]. For the models exploiting our TR scheduling technique, we fix the TR momentum constant m to 0.99 for simplicity. We choose the TR factor λ in a set of {5e-3, 1e-3, 5e-4, 1e-4}. When we train a scale parameter s in Eq. (13) of the main paper, we set the LR to the value ten times smaller than the one for weight parameters (*i.e.*, the LR in Table S8), similar to the works of [21, 26], for simplicity, instead of using the heuristic gradient scaling in LSQ [10]. Note that when we apply our TR scheduling technique, we do not train the scale parameters s for weight quantizers and fix the initial values, since training them makes it difficult to control a TR of quantized weights. We will discuss this in more detail in the following section.

S5. Detailed Description of a Quantizer

In this section, we explain the quantization process for a single quantizer. For simplicity, we consider a 2-bit weight quantizer based on Eq. (13) of the main paper, and use latent weights randomly drawn from the zero-mean Gaussian distribution with a standard deviation of 0.1. In the following,

Table S8. Hyperparameter settings in our experiments.

Dataset	Architecture	Optimizer	Epoch	Batch size	LR	Weight decay	Decay scheduler	TR factor λ	TR momentum m
CIFAR-10/100	ResNet-20 (ReActNet-18)	SGD	400 (200)	256	1e-1	1e-4	cosine	-	-
		SGDT						5e-3	0.99
		Adam			1e-3			-	-
		AdamT						5e-3	0.99
ImageNet	ResNet-18	SGD	120	256	1e-2	1e-4	cosine	-	-
		SGDT						1e-3	0.99
		Adam			1e-4			-	-
		AdamT						1e-3	0.99
	ReActNet-18	SGD	120	256	1e-1	0	linear	-	-
		SGDT						5e-4	0.99
		Adam			1e-3			-	-
		AdamT						5e-4	0.99
	MobileNetV2	SGD	120	256	1e-2	2.5e-5	cosine	-	-
		SGDT						1e-3	0.99
		Adam			1e-4			-	-
		AdamT						1e-3	0.99
	DeiT-T/S	AdamW	300	256	3e-4	5e-2	cosine	-	-
		AdamWT						1e-3	0.99
MS COCO	RetinaNet w/ ResNet backbone	SGD	90K (iter.)	16	1e-2	1e-4	cosine	-	-
		SGDT						1e-3	0.99

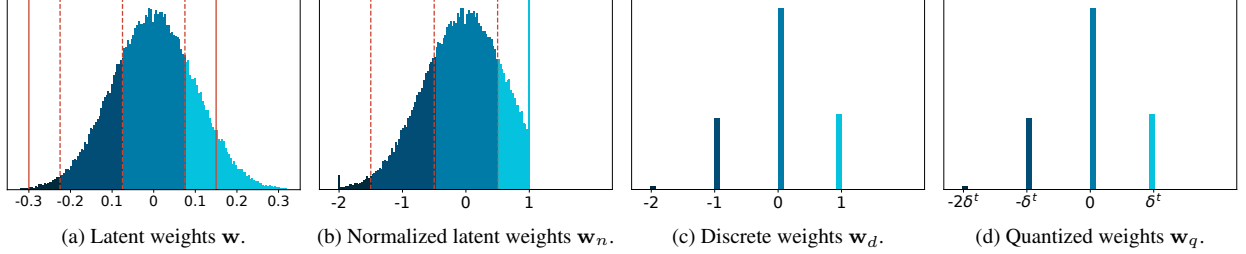


Figure S6. Visualization of the quantization process for a 2-bit weight quantizer. Solid vertical lines in (a) indicate clipping points w.r.t latent weights, which adjust a quantization interval. Dashed vertical lines in (a) and (b) represent transition points of the quantizer and a discretization function (*i.e.*, a round function) w.r.t the latent weights and normalized ones, respectively. The weights with the same color belong to the same discrete level of the quantizer. We denote by δ^t in (d) the distance between adjacent discrete levels of the quantizer at the t -th iteration step. (Best viewed in color.)

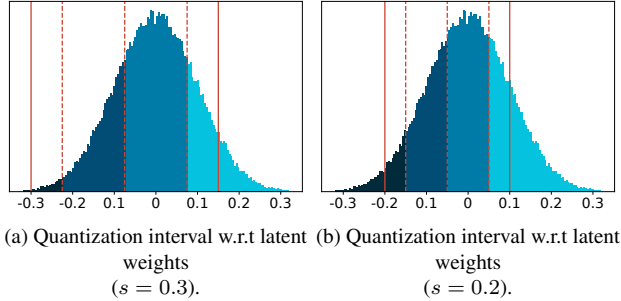


Figure S7. Comparison between quantization intervals for the same latent weights using different scale parameters. Solid and dashed vertical lines represent clipping and transition points of quantizers, respectively. The weights with the same color belong to the same discrete level of the quantizer. (Best viewed in color.)

we describe a detailed procedure of quantization according to the steps described in Eqs. (1)-(3) of the main paper. We visualize in Fig. S6 an overall quantization process for the quantizer. First, the latent weights $\mathbf{w}$ (Fig. S6a) are normalized and clipped by a normalization function, producing normalized latent weights $\mathbf{w}_n$ (Fig. S6b):

$$\mathbf{w}_n = \text{clip}\left(\frac{\gamma \mathbf{w}}{s}, \alpha, \beta\right), \quad (\text{S4})$$

where we set $\alpha = -2$, $\beta = 1$, and $\gamma = 2$ for 2-bit weight quantization. s is a learnable scale parameter adjusting a quantization interval. In Fig. S6, we set $s = 0.3$ for the purpose of visualization. Second, the normalized latent weights $\mathbf{w}_n$ are converted to discrete ones $\mathbf{w}_d$ (Fig. S6c) using a round function:

$$\mathbf{w}_d = \lceil \mathbf{w}_n \rceil. \quad (\text{S5})$$

Lastly, the discrete weights $\mathbf{w}_d$ are fed into a de-normalization function that applies post-scaling and outputs quantized weights $\mathbf{w}_q$ (Fig. S6d):

$$\mathbf{w}_q = \frac{1}{\gamma} \mathbf{w}_d. \quad (\text{S6})$$

Note that the post-scaling in Eq. (S6) is fixed, dividing the discrete weights with a constant value γ of 2. In this case, the distance between the adjacent discrete levels of the quantizer (*i.e.*, δ^t in Fig. S6d) is fixed for all training iterations, *i.e.*, $\delta^t = 0.5$ for all t .

S5.1. Counting transitions

In Eq. (5) in the main paper, we count the number of transitions by observing whether discrete weights, *i.e.*, integer numbers resulting from a round or a signum function (*e.g.*, $\mathbf{w}_d$ in Eq. (S5)), are changed or not after a single parameter update. As an example, suppose a case that a quantized weight at the t -th iteration step w_q^t belongs to the first level of the quantizer, *e.g.*, $w_q^t = -2\delta^t$ in Fig. S6d, where corresponding discrete weight w_d^t in Fig. S6c is -2 . If the quantized weight transits from the first to the second level of the quantizer after a parameter update (*i.e.*, $w_q^{t+1} = -\delta^{t+1}$), we can detect the transition using the discrete weight, since it is changed from -2 to -1 . Similarly, if the quantized weight remains in the same level after a parameter update (*i.e.*, $w_q^{t+1} = -2\delta^{t+1}$), we can say that the transition does not occur, because the discrete weight retains the same value. Note that we could use quantized weights $\mathbf{w}_q$ instead of discrete weights $\mathbf{w}_d$ for counting the number of transitions in Eq. (5) in the main paper, only when δ^t is fixed for all training iterations (*e.g.*, as in our quantizer in Eq. (13) of the main paper). Otherwise this could be problematic. For example, even if a quantized weight does not transit the discrete level after a parameter update, *e.g.*, $w_q^t = -2\delta^t$ and $w_q^{t+1} = -2\delta^{t+1}$, the quantized weight can be changed if δ^t and δ^{t+1} are not the same. This indicates that we cannot detect a transition with the condition of $\mathbb{I}[w_q^{t+1} \neq w_q^t]$, since the statement ($w_q^{t+1} \neq w_q^t$) could be always true, regardless of whether a transition occurs or not, if $\delta^{t+1} \neq \delta^t$ for all training iterations. Consequently, we count the number of transitions using discrete weights in Eq. (5) in the main paper, which is valid for general quantizers.

S5.2. Quantization interval

Following the recent state-of-the-art quantization methods [10, 21, 26], we introduce in Eq. (13) of the main paper (or in Eq. (S4)) a learnable scale parameter s . Given that α , β and γ in Eq. (13) of the main paper are bit-specific constants, the scale parameter s is the only factor that controls a quantization interval (*i.e.*, a clipping range) w.r.t quantizer inputs. We can see from Fig. S6a that the scale parameter ($s = 0.3$) is responsible for the positions of clipping

points w.r.t latent weights (solid vertical lines in Fig. S6a). It also decides transition points accordingly (dashed vertical lines in Figs. S6a), since the points are set by splitting the clipping range uniformly. This suggests that different scale parameters would give different sets of transition points. To verify this, we compare in Fig. S7 the quantization intervals using different scale parameters s for the same latent weights. We can see that the quantization interval shrinks if a smaller scale parameter ($s = 0.2$) is used, and the transition points are altered consequently. This again suggests that transitions could occur if the scale parameter s is changed during training, even when the latent weights are not updated. For example, a latent weight of -0.2 in Fig. S7a belongs to the second level of the quantizer, whereas that in Fig. S7b belongs to the first level. Within our TR scheduling technique, we attempt to control the number of transitions by updating latent weights with a TALR, but the transitions could also be affected by the scale parameter. For this reason, we do not train the scale parameters in weight quantizers, when the TR scheduling technique is adopted, fixing the transition points of the quantizers and controlling the transitions solely with the latent weights.